

Projekt SWTM

Dokumentation

Aktivitätstracker

Android Application

Auftraggeber: pep.digital GmbH, Esslingen

Modul: Projektarbeit SWTM

Hochschule: Hochschule Esslingen

Semester: Sommersemester 2026

Teammitglieder

Name	Matrikelnummer
Daniel Trbara	779034
Christian Sakru	779980
Lionel Toussoumsnaba	781030
Mirac Sancak	781232
Abdullah Bilen	779202

Inhaltsverzeichnis

1	Meilenstein	1
1.1	Problemstellung und Zielgruppe	1
1.1.1	Problemstellung	1
1.1.2	Zielgruppe	1
1.2	Geforderter Funktionsumfang	2
1.2.1	Minimum Viable Product (MVP)	2
1.2.2	Erweiterte Ziele (Extended Goals)	2
1.2.3	Zielsetzung	3
1.3	UI-Entwürfe	4
1.4	Architektur (konzeptionell + technologisch)	9
1.4.1	Logische Sicht	9
1.4.2	Technischer Stack	9
1.4.3	Begründung der Technologiewahl	10
1.5	Projektmanagement	11
1.5.1	Ziel des Projektmanagements	11
1.5.2	Arbeitsweise	11
1.5.3	Kommunikations- und Organisationstools	12
1.6	Aufwandsschätzung	12
2	Meilenstein	14
2.1	User Stories	14
2.1.1	MVP	14
2.1.2	Extended	16
2.2	Die Softwarearchitektur und ihre logische Sichten	17
2.2.1	Die Struktursicht	17
2.2.2	Verhaltenssicht	18
2.2.3	Verteilungssicht	21
2.3	Verwendete Technologien und Frameworks	22
2.4	Verwendete Schnittstellentechnologie	23
2.5	Datenbank	26
2.5.1	Logisches Datenmodell	26
2.5.2	Physisches Datenmodell	27
3	Meilenstein	28
3.1	Installations- und Administrationshandbuch	28
3.1.1	Installation für Entwickler	28
3.1.2	Installation für Endnutzer	29
3.1.3	Administration	29
3.2	Aufteilung des Teams	30

3.3	Reflektion Projektmanagement	31
3.4	Reflektion Lernfortschritt	32
3.5	Ausblick	33
Anhang		34
A.1	Lizenzen	35
A.1.1	Verwendete Frameworks	35
A.1.2	Verwendete Bibliotheken	35
A.1.3	Lizenzkompatibilität	37
A.1.4	Lizenz des eigenen Codes	37
Quellen		38

1 Meilenstein

1.1 Problemstellung und Zielgruppe

1.1.1 Problemstellung

Viele Menschen verbringen einen Großteil ihres Alltags sitzend, sei es im Büro oder im Home-Office vor dem PC. Bewegung kommt dabei häufig zu kurz. Vor allem in den Wintermonaten verstärkt sich dieses Problem durch den Mangel an Sonnenlicht. Langes Sitzen ohne ausreichende Bewegung führt zu einer schlechten Körperhaltung, die langfristig die Muskulatur schwächt.

Kernproblem: Büroangestellte und Home-Office-Arbeitende bewegen sich im Alltag zu wenig, was zu gesundheitlichen Langzeitfolgen führt. Es fehlt ein niedrigschwelliges, motivierendes Werkzeug zur Integration von Alltagsbewegung.

Hier setzt der **Aktivitätstracker** an: Er soll Nutzerinnen und Nutzer motivieren, regelmäßig Outdoor-Aktivitäten durchzuführen. Dies wird durch ein Streak-System und tagesbasierte Erinnerungen unterstützt.

1.1.2 Zielgruppe

Die Anwendung richtet sich primär an folgende Nutzergruppe:

- Personen, die überwiegend sitzend im Büro oder Home-Office arbeiten
- Nutzer, die fit und gesund bleiben möchten, mit leichter regelmäßiger Outdoor-Aktivität (z. B. Spazierengehen, Radfahren)
- Die Zielgruppe ist **nicht** leistungsorientiert

1.2 Geforderter Funktionsumfang

1.2.1 Minimum Viable Product (MVP)

Das MVP beschreibt die grundlegenden Funktionen, die notwendig sind, um einen ersten nutzbaren Prototyp bereitzustellen.

Funktion	Beschreibung
Aktivitäts-Tracking	Nutzer können tägliche Aktivitäten (z. B. Gehen, Radfahren) erfassen.
Streak-System	Das System zählt aufeinanderfolgende aktive Tage. Bei fehlender Aktivität wird die Streak unterbrochen. Dient als Motivationsmechanismus.
Widgets	Home-Screen-Widgets ermöglichen schnelle Eingabe und direkte Streak-Anzeige ohne App-Öffnung.
Wetter/tagesbasierte Erinnerungen	Push-Benachrichtigungen basierend auf Uhrzeit oder Wetterbedingungen motivieren zur Aktivität bei günstigen äußeren Bedingungen.

1.2.2 Erweiterte Ziele (Extended Goals)

Die folgenden Funktionen erweitern den MVP und verbessern die Benutzererfahrung. Sie sind für die erste Version nicht zwingend erforderlich.

- **Statistiken:** Längste Streak, Wochen-/Monatsübersichten, Aktivitätsanalysen
- **Dark-/Light-Theme:** Umschaltbares Design nach Nutzerpräferenz
- **Mehrsprachigkeit:** Deutsch und Englisch als primäre Sprachen
- **Streaks mit Freunden teilen:** Soziale Komponente zur gegenseitigen Motivation
- **Öffentliche Challenges:** Gruppenherausforderungen zur Steigerung der Motivation durch Wettbewerb

1.2.3 Zielsetzung

Der Aktivitätstracker verfolgt konkrete, messbare Ziele, die über die allgemeine Problemlösung hinausgehen und als Erfolgskriterien für das Projekt dienen:

- Nutzer sollen jeden Tag eine Outdoor-Aktivität erfassen können
- Eine Aktivität soll innerhalb von wenigen Sekunden erfassbar sein
- Wetter- und tagesbasierte Benachrichtigungen sollen automatisch bei geeigneten Bedingungen ausgelöst werden
- Das Streak-System soll die tägliche Nutzungsfrequenz messbar erhöhen, ohne dabei leistungsorientierten Druck aufzubauen

Auf Basis dieser Ziele wurden die folgenden Architektur- und Technologieentscheidungen getroffen, die in Abschnitt [1.4](#) beschrieben werden.

1.3 UI-Entwürfe

Die folgenden Abbildungen zeigen die Entwürfe der Anwendung. Die Bildunterschriften sind jeweils mit dem Quellenverzeichnis verlinkt. Alle UI-Entwürfe wurden von Lionel Toussoumsnaba in Figma entwickelt (siehe Quelle [\[1\]](#)).

Der Hauptbildschirm (siehe Abbildung 1.1) zeigt die aktuelle Streak-Anzeige des Nutzers. Die Streak-Visualisierung ist von modernen Tracking-Apps wie YAZIO oder Duolingo inspiriert.

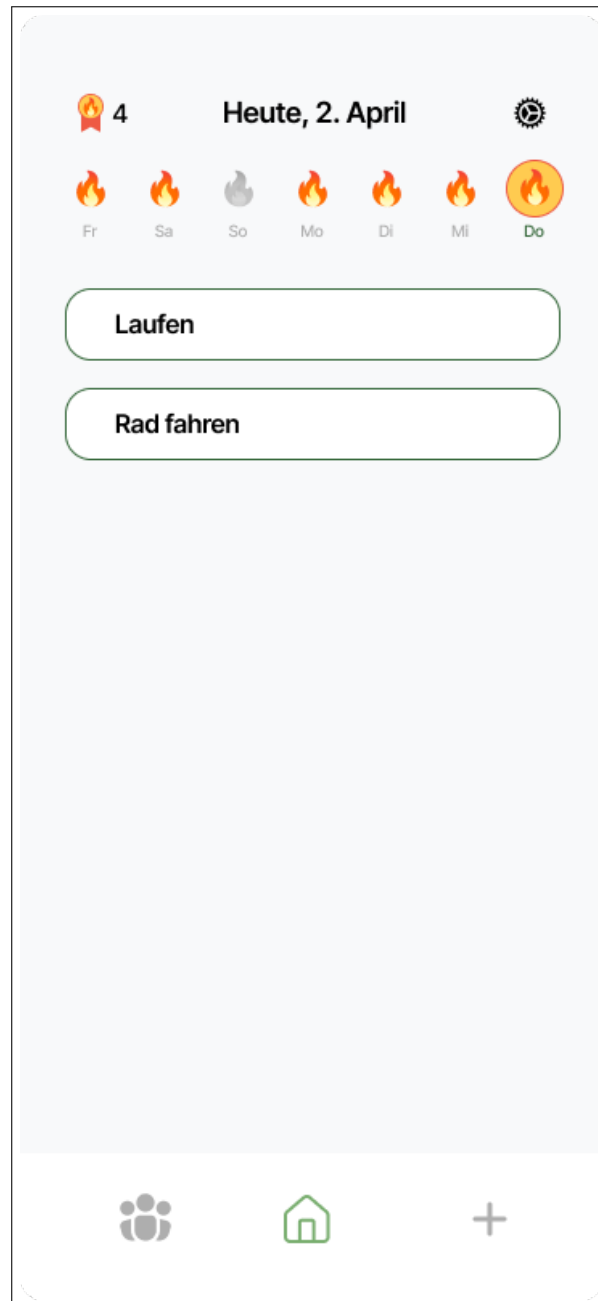


Abbildung 1.1: [Home Screen: Hauptmenü mit Streak-Anzeige](#)

Die Registrierung (siehe Abbildung 1.2) ermöglicht die erstmalige Anmeldung per E-Mail oder alternativ über ein Google-Konto. Der Anmeldebildschirm (siehe Abbildung 1.3) bietet dieselben Login-Optionen für bereits registrierte Nutzer.

The registration screen features the 'Steady' logo at the top. Below it, the heading 'Registrierung' is followed by the text 'Willkommen! Registriere dich, um fortzufahren.' The form includes three input fields: 'Username', 'E-Mail', and 'Passwort'. Below these fields, there is a button with the Google logo and the text 'Mit Google anmelden'. At the bottom, there is a large green button labeled 'Registrieren'.

Abbildung 1.2: [Registrierung: Übersicht zur erstmaligen Anmeldung oder alternativen Google-Login](#)

The login screen features the 'Steady' logo at the top. Below it, the heading 'Anmeldung' is followed by the text 'Melde dich an, um fortzufahren.' The form includes two input fields: 'E-Mail' and 'Passwort'. Below these fields, there is a button with the Google logo and the text 'Mit Google anmelden'. At the bottom, there is a large green button labeled 'Anmelden'.

Abbildung 1.3: [Anmeldebildschirm: Login via E-Mail und Passwort oder alternatives Google-Login](#)

Neue Aktivitäten können über ein Modal erfasst werden (siehe Abbildung 1.4). Bestehende Einträge lassen sich anschließend über einen separaten Bearbeitungsscreen anpassen (siehe Abbildung 1.5).

Das Modal 'Aktivität erstellen' hat einen Titel mit einem Schließen-Symbol (X) und dem Text 'Aktivität erstellen'. Es enthält zwei Eingabefelder: 'Name' mit dem Wert 'Ins Gym gehen' und 'Datum' mit dem Wert '08.06.2026'. Am unteren Rand befindet sich ein grüner Button mit der Aufschrift 'Speichern'.

Abbildung 1.4: Aktivität erstellen: Modal zur Eingabe und Speicherung einer neuen Aktivität

Das Modal 'Aktivität bearbeiten' hat einen Titel mit einem Schließen-Symbol (X) und dem Text 'Aktivität bearbeiten'. Es enthält zwei Eingabefelder: 'Name' mit dem Wert 'Rad fahren' und 'Datum' mit dem Wert '08.06.2026'. Am unteren Rand befindet sich ein grüner Button mit der Aufschrift 'Speichern'.

Abbildung 1.5: Aktivität bearbeiten: Modal zur Anpassung einer bereits bestehenden Aktivität

Im Einstellungsbereich (siehe Abbildung 1.6) können Nutzer Benachrichtigungszeiten und Kontodetails verwalten. Die Profileinstellungen (siehe Abbildung 1.7) ermöglichen zusätzlich die Änderung des Benutzernamens.

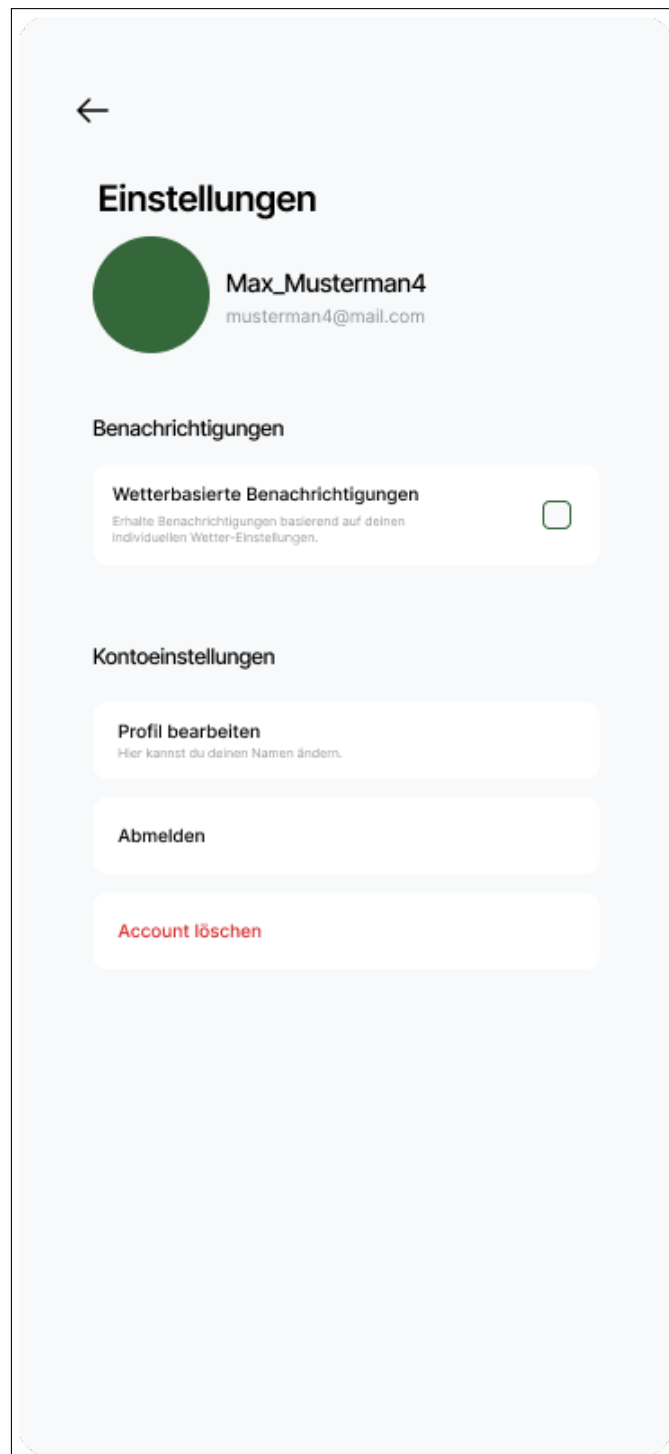


Abbildung 1.6: Einstellungen: Verwaltung von Benachrichtigungen und Kontodetails

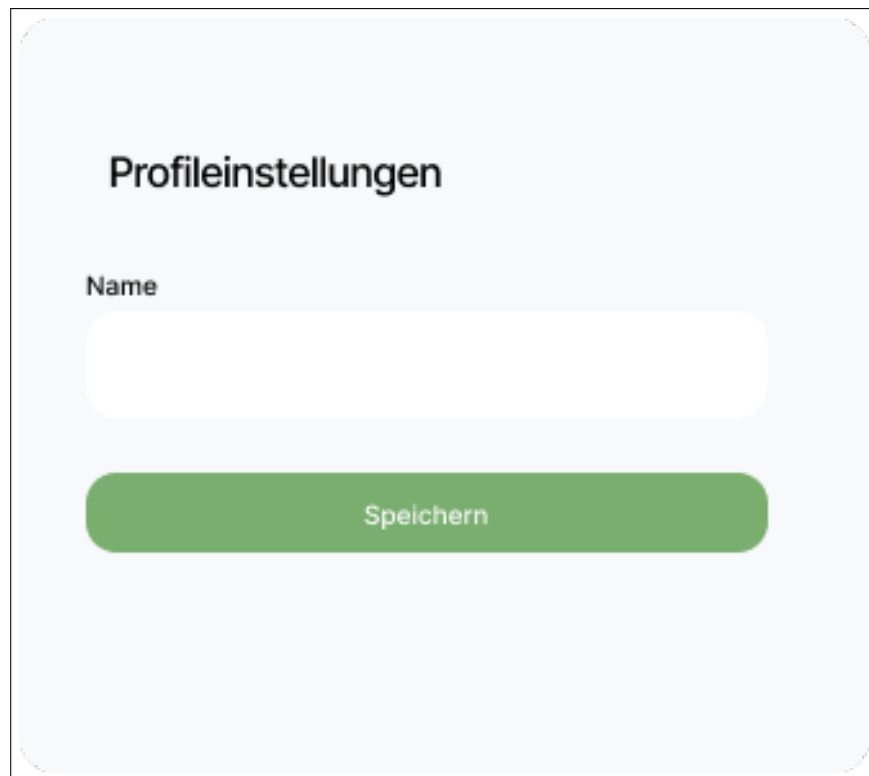


Abbildung 1.7: Profileinstellungen: Menü zur Änderung des Benutzernamens

Push-Benachrichtigungen (siehe Abbildung 1.8) informieren den Nutzer bei Inaktivität sowie bei erreichten Streak-Meilensteinen. Das Home-Screen-Widget (siehe Abbildung 1.9) ermöglicht einen schnellen Überblick über die wöchentliche Streak ohne die App öffnen zu müssen.

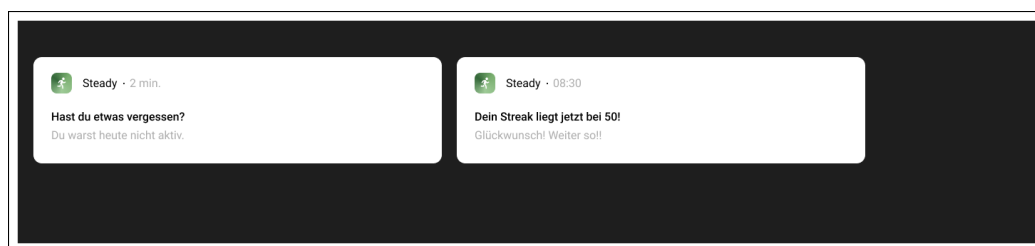


Abbildung 1.8: Benachrichtigungen: Darstellung von Inaktivitäts-Erinnerungen und Streak-Meilensteinen

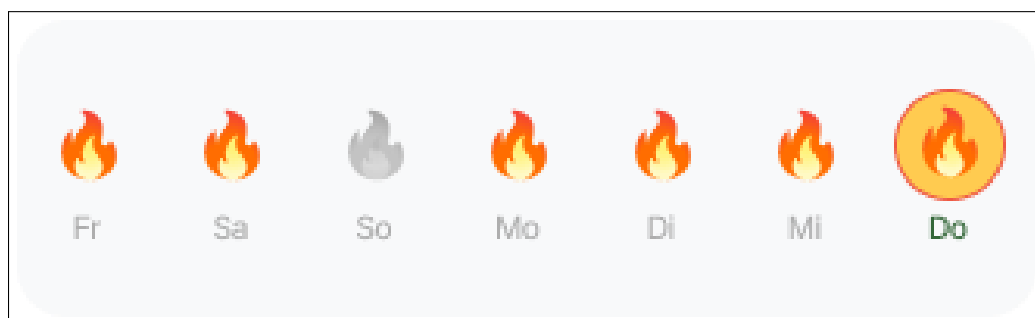


Abbildung 1.9: Streak-Widget: Isolierte Ansicht der wöchentlichen Streak-Visualisierung

1.4 Architektur (konzeptionell + technologisch)

1.4.1 Logische Sicht

Die Anwendung ist in drei Schichten aufgeteilt: UI-Schicht, Logik-Schicht und Datenzugriff. Wie in Abbildung 1.10 dargestellt, kommuniziert die UI-Schicht (Jetpack Compose) ausschließlich über die Logik-Schicht mit dem Datenzugriff. Der Datenzugriff kommuniziert sowohl mit der lokalen Room-Datenbank als auch mit dem Spring Boot Backend und der WeatherAPI über HTTP.

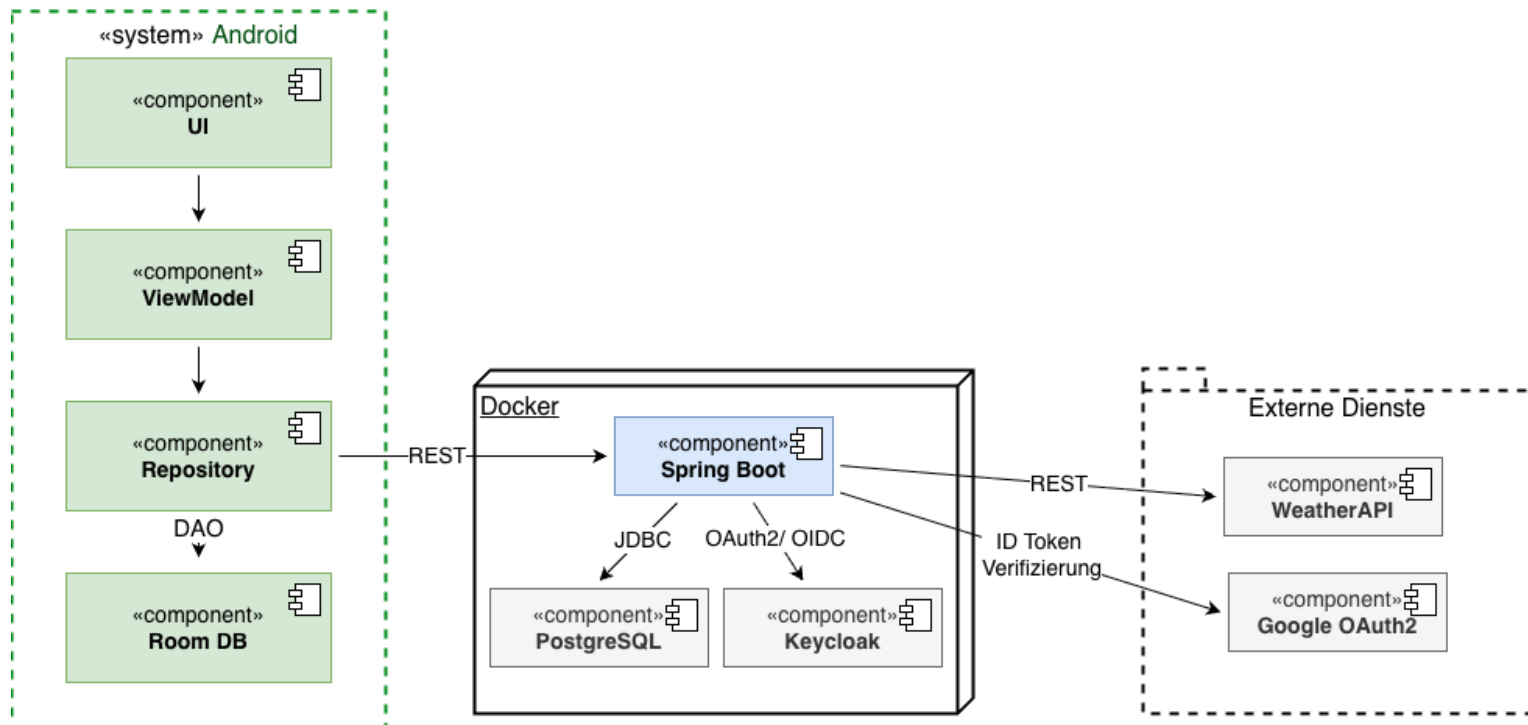


Abbildung 1.10: Komponentenmodell

1.4.2 Technischer Stack

Der technische Stack (siehe Tabelle 1.1) umfasst alle eingesetzten Technologien und Bibliotheken der Anwendung. Die Begründung der einzelnen Technologieentscheidungen ist in Abschnitt 1.4.3 beschrieben.

Schicht	Technologie / Bibliothek
Frontend	Android App + Jetpack Compose (Kotlin)
Architektur	Schichtenbasierter Ansatz (UI, Logik, Datenzugriff)
Lokale DB	Room (SQLite) – Offline-Verfügbarkeit
API-Client	Retrofit 2 – REST-Kommunikation
Backend	Spring Boot – REST API und Datenverwaltung
Authentifizierung	Keycloak
Datenbank	PostgreSQL
Wetter-API	WeatherAPI.com – wetterbasierte Erinnerungen
Widgets	Glance API (Jetpack)
Versionskontrolle	GitHub ¹
Build-System (Android)	Gradle (Kotlin DSL)
Build-System (Backend)	Maven

Tabelle 1.1: Technischer Stack des Projekts

1.4.3 Begründung der Technologiewahl

Die Auswahl der eingesetzten Technologien basiert sowohl auf Vorgaben des Auftraggebers als auch auf technischen und wirtschaftlichen Überlegungen des Teams.

Keycloak wurde als Authentifizierungslösung gewählt, da es vom Auftraggeber pep.digital und der Hochschule vorgegeben wurde und den OAuth2/OIDC-Standard vollständig implementiert.

PostgreSQL wird serverseitig als primäre Datenbank eingesetzt, da Nutzerdaten und Aktivitäten zentral gespeichert werden müssen, um geräteübergreifenden Zugriff zu ermöglichen. Room dient lediglich als lokaler Cache für Offline-Verfügbarkeit und als Ergänzung zu PostgreSQL.

Spring Boot wurde als Backend-Framework gewählt, da es vorgegeben wurde und für die Tracking-App mit den anderen eingesetzten Technologien kompatibel ist.

Jetpack Compose ist das moderne UI-Framework für Android und wird von Google als offizieller Nachfolger des klassischen XML-basierten View-Systems empfohlen. Es wurde vom Auftraggeber vorgegeben und ermöglicht eine deklarative UI-Entwicklung, bei der die Oberfläche direkt den aktuellen Zustand der Anwendung widerspiegelt. Gegenüber dem klassischen View-System entfällt unnötiger Overhead im Code.

Glance API wurde für die Umsetzung der Widgets gewählt, da sie eine moderne und deklarative Entwicklung von Android-Widgets ermöglicht. Die API reduziert den Implementierungsaufwand und verbessert die Wartbarkeit des Codes. Zudem passt

¹Projekt-Repository: <https://github.com/hse-classes/ss26-aktivitatstracker>

Glance gut zum Jetpack-/Compose-Ansatz und eignet sich besonders für einfache, schnell zugängliche Informationen wie Aktivitäten und Streaks.

Retrofit ist bekannt dafür, Netzwerkanfragen für Android- und Java-Entwicklungen in wenigen Zeilen Code abzubilden. Es wurde ausgewählt, um die Verwaltung von REST-Calls zu vereinfachen und den Code lesbarer zu gestalten.

WeatherAPI.com wurde ausgewählt, da der kostenlose Tarif mit 100.000 Anfragen pro Monat für den Projektumfang ausreichend ist. Die Integration erfolgt via Retrofit, wobei der API-Key sicher im Backend verwaltet wird. Ein Vorteil ist die Rückgabe der Daten als strukturiertes JSON, was eine direkte Verarbeitung im Projekt ermöglicht (siehe Quelle [2]).

Room wurde gewählt, da es als offizieller Google-Standard gut dokumentiert ist und Datenbankfehler bereits beim Kompilieren erkennt. Die direkte Umwandlung von Datenbankeinträgen in Kotlin-Objekte reduziert den Implementierungsaufwand und verhindert typische Laufzeitfehler.

1.5 Projektmanagement

1.5.1 Ziel des Projektmanagements

Ziel ist eine strukturierte und nachvollziehbare Projektdurchführung in agilen Ein-Wochen-Sprints. Durch regelmäßige Meetings und konsequentes Issue-Tracking soll sichergestellt werden, dass Termine eingehalten und Fortschritte transparent dokumentiert werden.

1.5.2 Arbeitsweise

Das Team arbeitet nach einem Issue-basierten Ansatz ohne fest zugewiesene Rollen. Aufgaben werden als Issues im GitHub-Repository angelegt und vom Team untereinander aufgeteilt. Dies fördert flexible Zusammenarbeit und gleichmäßige Beteiligung aller Mitglieder. Das Projekt-Repository ist unter folgendem Link erreichbar: <https://github.com/hse-classes/ss26-aktivitatstracker>

1.5.3 Kommunikations- und Organisationstools

Zur Koordination und Kommunikation im Team werden die in Tabelle 1.2 aufgeführten Tools eingesetzt.

Tool	Verwendungszweck
GitHub	Versionskontrolle, Issue-Tracking, Kanban-Board
Discord	Team-Kommunikation
Figma	UI-Design, Prototypen
Overleaf	Dokumentierung
Microsoft Teams	Meetings mit Betreuer und Auftraggeber

Tabelle 1.2: Kommunikations- und Organisationstools

1.6 Aufwandsschätzung

Die folgende Tabelle (siehe Tabelle 3.3) zeigt die geschätzten Aufwände. Das Projekt ist in **Ein-Wochen-Sprints** gegliedert. Die Schätzung basieren auf Refinements der MVP's. Die Schätzungen wurden gemeinsam im Team festgelegt und orientieren sich an den angelegten Issues im GitHub-Repository.

Phasen	User Story	Aufwandsschätzung
Projektmanagement & Vorbereitung		
Sprint-Meetings, Ticket-Refinement & Organisation	–	210 h
Anforderungsanalyse, Architektur & UI-Design	–	160 h
Implementierung		
Setup (Backend, Keycloak, Pipeline, Refactoring, Sync, APK, Server)	1.x, 2.x	133 h
Aktivitäten-Tracking	3.x	54 h
Streak-System	4.x	28 h
Benachrichtigungs-Service & Wetter-API	5.x, 6.x	48 h
Widget (Glance API)	7.x	32 h
Freunde	11.x, 12.x	25 h
Statistiken	8.x	16 h
Qualitätssicherung & Abschluss		
Code-Reviews, Bugfixing & Testing	–	220 h
Dokumentation (MS-Doku, Präsi & Handbuch)	–	260 h
Gesamtaufwand geschätzt (Team)		1.250 h

Tabelle 1.3: Aufwandsschätzung

2 Meilenstein

2.1 User Stories

2.1.1 MVP

Registrieren

1.1 Als Benutzer möchte ich mich einmalig Registrieren können, damit ich Geräte übergreifend über meinen Fortschritt zugreifen kann.

Anmelden

2.1 Als Benutzer möchte ich mich anmelden können, damit ich wieder Zugriff auf meinen Account habe.

2.2 Als Benutzer möchte ich meinen Account löschen, damit alle meine Daten gelöscht werden.

Aktivitäten Tracken

3.1 Als Benutzer möchte ich eine Aktivität eintragen, damit ich mein Verhalten tracken kann.

3.2 Als Benutzer möchte ich meine Aktivitäten bearbeiten können, damit ich Fehler korrigieren kann.

3.3 Als Benutzer möchte ich meine Aktivitäten löschen können, damit ich falsche Einträge entfernen kann.

3.4 Als Benutzer möchte ich eine Liste meiner Aktivitäten sehen, damit ich den Überblick behalte.

3.5 Als Benutzer möchte ich meine Aktivitäten auch ohne Internetverbindung aufzeichnen, damit ich auch in einem Funkloch meine Aktivitäten Tracken kann.

3.6 Als Benutzer möchte ich den Namen der Aktivität nicht immer wieder aufschreiben müssen, sondern meine vorherigen Aktivitäten benutzen.

Streak verwalten

4.1 Als Benutzer möchte ich meinen aktuellen Streak sehen, damit ich motiviert bleibe.

4.2 Als Benutzer möchte ich eine Streak aufbauen können, damit ich einen Ansporn habe.

4.3 Als Benutzer möchte ich meine Streak verlieren können, damit ich bestraft werde.

4.4 Als Benutzer möchte ich meinen Streak-Verlauf sehen, damit ich meinen Fortschritt nachvollziehen kann.

Erinnerungen erhalten

5.1 Als Benutzer möchte ich Erinnerungen bekommen, damit ich meine Aktivität nicht vergesse.

5.2 Als Benutzer möchte ich Erinnerungen konfigurieren, damit ich zur passenden Zeit die richtige Motivation bekomme.

5.3 Als Benutzer möchte ich meine Erinnerung ändern können, damit sie besser zu meinem Tagesablauf passt.

Wetter basierte Erinnerungen erhalten

6.1 Als Benutzer möchte ich eine Erinnerung bekommen, wenn es draußen gutes Wetter ist.

Widgets nutzen

7.1 Als Benutzer möchte ich Aktivitäten über ein Widget eintragen, damit es schneller geht.

7.2 Als Benutzer möchte ich Widgets hinzufügen, damit ich meine Streaks schneller sehen kann.

7.3 Als Benutzer möchte ich ein Widget entfernen können, damit ich meinen Homescreen anpassen kann.

2.1.2 Extended

Statistiken checken

8.1 Als Benutzer möchte ich Statistiken sehen, damit ich meinen Fortschritt analysieren kann.

8.2 Als Benutzer möchte ich meine Statistiken nach Woche oder Monat filtern, damit ich meinen Fortschritt besser analysieren kann.

8.3 Als Benutzer möchte ich meinen längsten Streak sehen, damit ich meine beste Leistung kenne.

Theme ändern

9.1 Als Benutzer möchte ich zwischen Dark und Light Mode wechseln, damit ich die App angenehmer nutzen kann.

Sprache ändern

10.1 Als Benutzer möchte ich die App in verschiedenen Sprachen nutzen, damit ich sie besser verstehe.

Freund Streak ansehen

11.1 Als Benutzer möchte ich in der App meine Freunde und ihre Streaks sehen.

11.2 Als Benutzer möchte ich Freunde hinzufügen können, damit ich mich mit ihnen verbinden kann.

11.3 Als Benutzer möchte ich Freunde entfernen können, damit ich meine Liste verwalten kann.

Streak mit Freund teilen

12.1 Als Benutzer möchte ich meinen Streak mit Freunden teilen, damit ich mich austauschen kann

2.2 Die Softwarearchitektur und ihre logische Sichten

2.2.1 Die Struktursicht

Die Strukturschichten beschreiben die internen Bausteine des Systems und deren Beziehungen zueinander.

Android App

Entwickelt wird die App mit Android Studio's Jetpack Compose; ein UI-Framework für die Darstellung der Nutzeroberfläche und Nutzerinteraktion.

ViewModel

Beinhaltet die Geschäftslogik, beispielsweise die Aktivitätsverwaltung und Streakberechnung.

Repository

Kapselt den Datenzugriff und fungiert als Schnittstelle zwischen der Logik-Schicht und den Datenbanken.

RoomDB

Dient als lokale SQLite-Datenbank zur Offline-Verfügbarkeit, auf die über Data Access Objects (DAO) zugegriffen wird.

Backend-Systeme

Spring Boot

Stellt das Backend dar, welches über REST angesprochen wird und die zentrale Datenverwaltung übernimmt.

PostgreSQL

Dient als relationale Datenbank zur persistenten und serverseitigen Speicherung. Die Anbindung an das Spring Boot Backend erfolgt über JDBC.

Keycloak

Ist für die Authentifizierung mittels OAuth2/OIDC-Standards zuständig.

Die genaue Anordnung und Kommunikation dieser gesamten Bausteine ist in Abbildung [2.1](#) visualisiert.

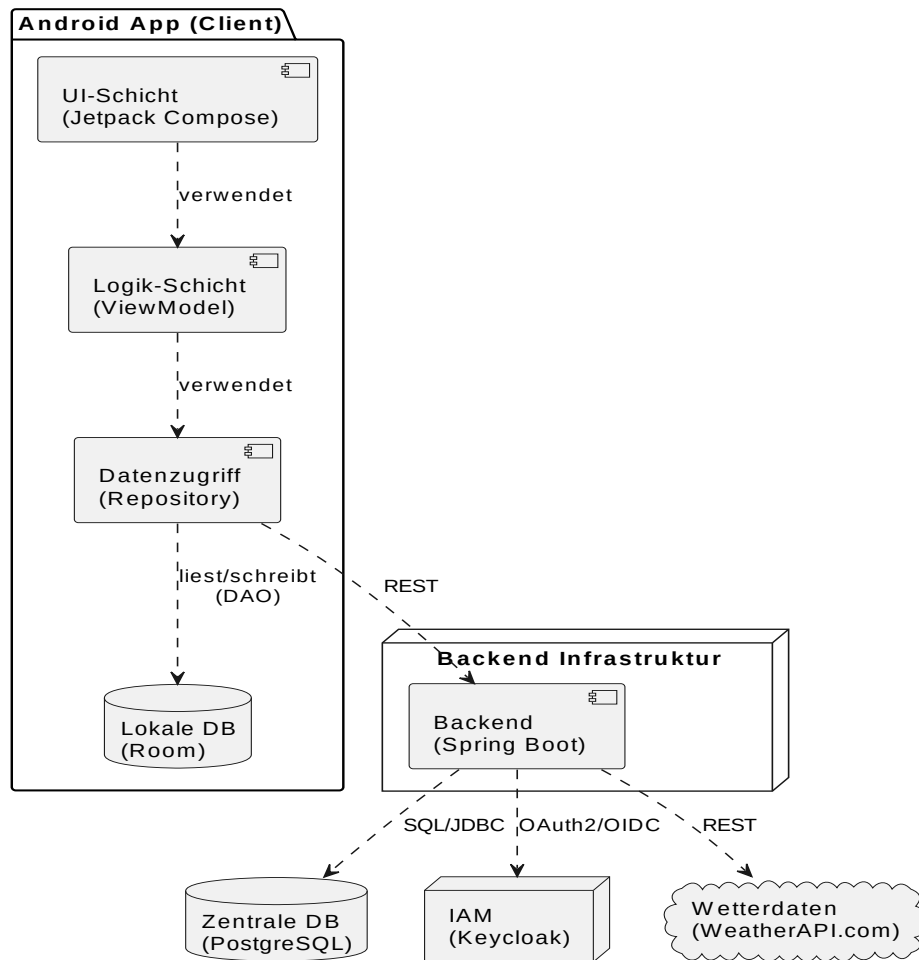


Abbildung 2.1: Komponentendiagramm der Strukturschicht

2.2.2 Verhaltenssicht

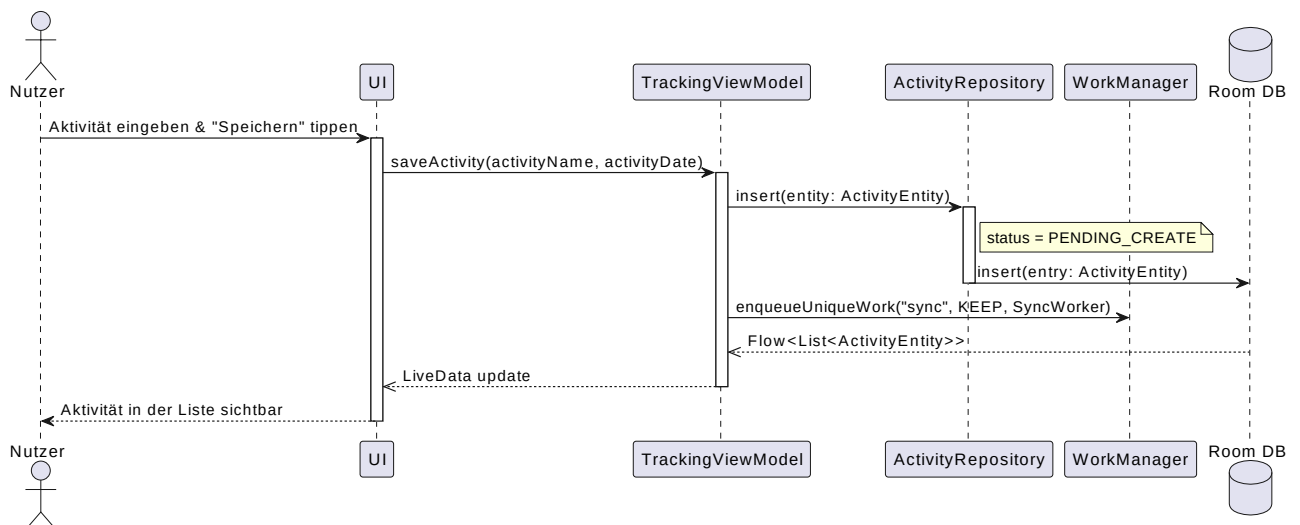


Abbildung 2.2: Prozess der Aktivitätserfassung

Die Grafik 2.2 veranschaulicht, wie die Softwarekomponenten zur Laufzeit miteinander interagieren, um die Kernanwendungsfälle abzuarbeiten.

Aktivität erfassen und speichern

Der Nutzer gibt eine neue Aktivität über ein Eingabefeld in der UI-Schicht ein. Die UI leitet die Eingabedaten an das TrackingViewModel weiter, welches das Repository aufruft. Das Repository erzeugt aus der Entity einen Eintrag mit dem Status PENDING_CREATE und speichert diesen über das DAO in der lokalen Room-Datenbank.

Dies gewährleistet eine sofortige Speicherung und Offline-Verfügbarkeit. Anschließend plant das ViewModel über den WorkManager einen Synchronisationsauftrag ein (enqueueUniqueWork mit ExistingWorkPolicy.KEEP, sodass parallele Sync-Läufe vermieden werden).

Nach dem Speichern aktualisiert Room über einen reaktiven Datenstrom (Flow, im ViewModel als LiveData bereitgestellt) automatisch die UI, sodass die neue Aktivität dem Nutzer unmittelbar angezeigt wird.

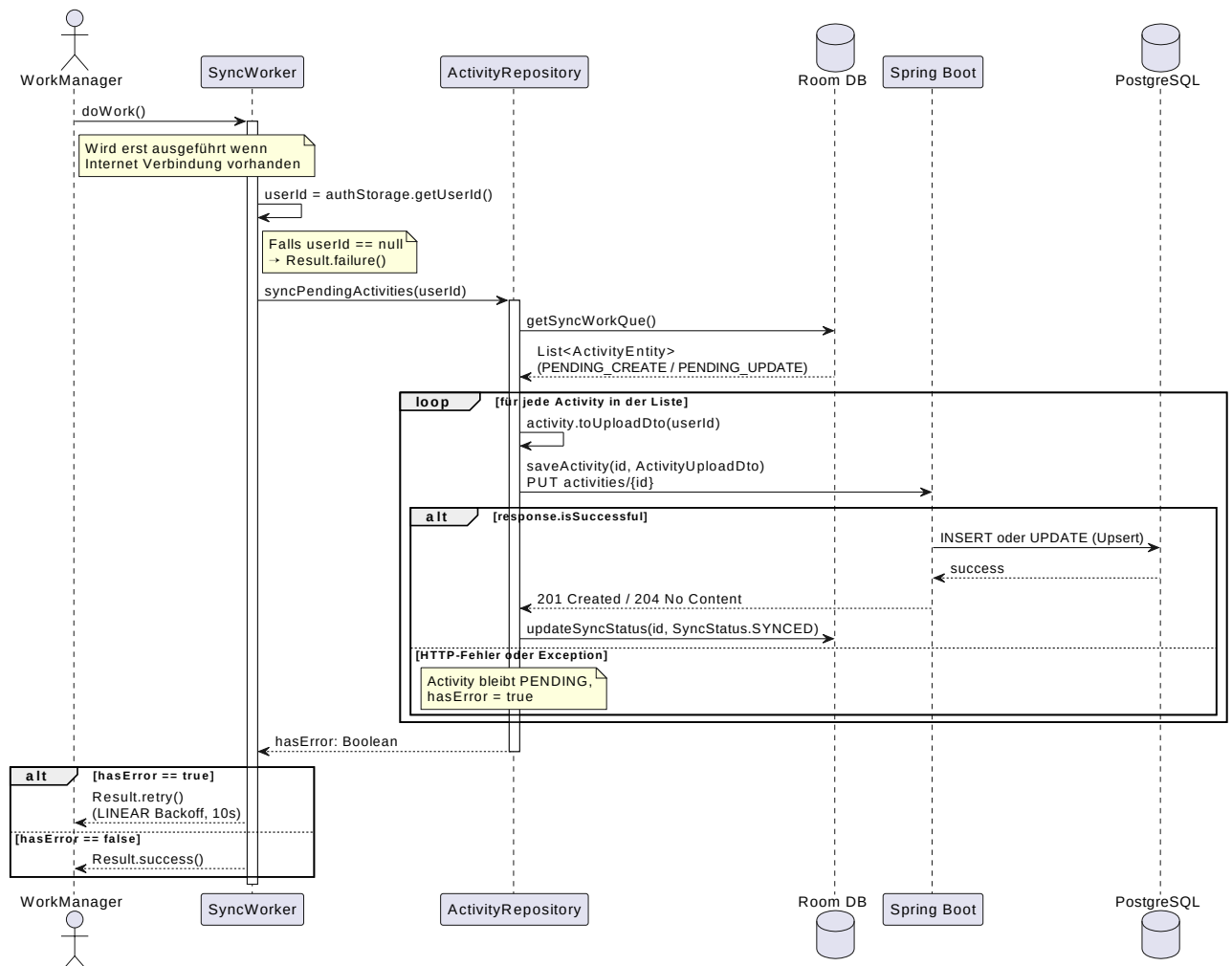


Abbildung 2.3: Prozess der Aktivitätssynchronisierung

Die Grafik 2.3 veranschaulicht, wie die Softwarekomponenten interagieren, um Aktivitäten synchronisieren zu können

Aktivitäten synchronisieren

Der WorkManager startet den SyncWorker, sobald eine Netzwerkverbindung verfügbar ist. Dieser ermittelt zunächst die userId (bei fehlender ID bricht der Worker mit Result.failure() ab) und ruft anschließend das Repository auf, welches alle noch nicht synchronisierten Einträge (Status PENDING_CREATE oder PENDING_UPDATE) aus der lokalen Room Datenbank lädt.

Für jeden dieser Einträge wird die Entity in ein ActivityUploadDto umgewandelt und per PUT /activities/id an das Spring Boot Backend gesendet. Da Room die lokale Single Source of Truth ist und die id bereits dort vergeben wird, erfolgt die Übertragung als PUT auf diese feste id (Upsert) statt als POST. Das Backend legt den Datensatz in der PostgreSQL Datenbank an (201 Created) bzw. aktualisiert einen bereits vorhandenen Eintrag (204 No Content). Der Aufruf ist über die feste id idempotent. Bei erfolgreicher Übertragung wird der lokale Status auf SYNCED gesetzt.

Schlägt ein Request fehl (HTTP-Fehler oder Exception), bleibt der betroffene Eintrag auf PENDING und es wird hasError = true gesetzt. In diesem Fall liefert der SyncWorker Result.retry() zurück, sodass der WorkManager den gesamten Sync-Lauf mit linearem Backoff (10 s) erneut einplant; andernfalls endet er mit Result.success(). Eine weitergehende Fehlerbehandlung war vom Kunden nicht erwünscht.

Authentifizierung

Versucht der Nutzer sich über die App anzumelden (z. B. via E-Mail/Passwort oder Google-Login), kommuniziert das System mit Keycloak. Keycloak verifiziert die Identität per OAuth2/OIDC-Protokoll und autorisiert den sicheren Zugriff auf das Backend.

2.2.3 Verteilungssicht

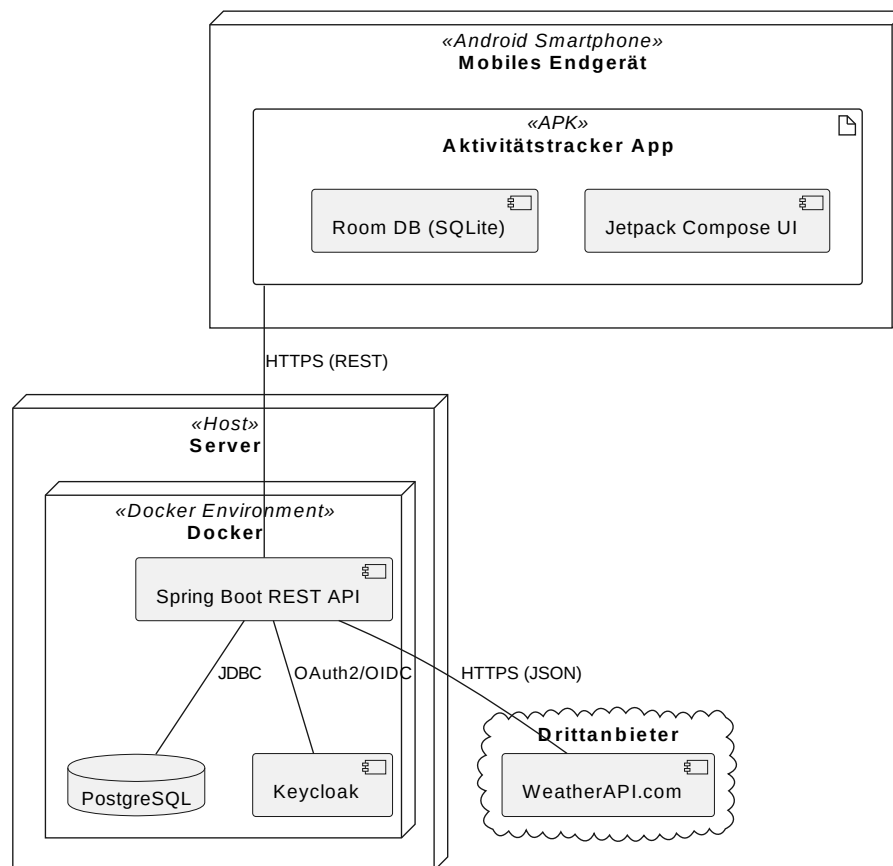


Abbildung 2.4: Deployment-Diagramm der Verteilungssicht

Die in Abbildung 2.4 abgebildete Verteilungssicht beschreibt die physische Verteilung der Softwarekomponenten auf die Hardware- und Netzwerk-Infrastruktur.

Client-Gerät (Android-Smartphone)

Auf dem Endgerät des Nutzers wird die kompilierte Android Application ausgeführt. Dieser Knoten beinhaltet das Frontend (Jetpack Compose) sowie die Room-Datenbank, welche als Single Source of Truth dient und Offline-Verfügbarkeit ermöglicht.

Server-Infrastruktur

Die serverseitigen Komponenten werden in einer gemeinsamen Docker-Umgebung auf einem einzelnen Server betrieben:

- **Spring Boot** stellt die zentrale REST-API für die App bereit.
- **PostgreSQL** speichert alle Nutzerdaten und Aktivitäten geräteübergreifend.
- **Keycloak** übernimmt die Authentifizierung mittels OAuth2/OIDC.

Zusätzlich werden über die externe Schnittstelle WeatherAPI.com Wetterdaten als strukturiertes JSON bezogen, um wetterbasierte Erinnerungen zu ermöglichen.

Die Kommunikation zwischen Android-App und Backend erfolgt momentan über HTTPS. In der aktuellen Entwicklungsumgebung werden selbstsignierte Zertifikate eingesetzt.

2.3 Verwendete Technologien und Frameworks

Im folgenden Abschnitt werden die verwendeten Technologien und Frameworks beschrieben, die im Projekt eingesetzt werden oder deren Einsatz geplant ist.

Frontend (Android)

Die Android-App wird vollständig in Kotlin entwickelt. Folgende Frameworks kommen zum Einsatz:

Jetpack Compose

Jetpack Compose wird als UI-Framework eingesetzt, da es vom Auftraggeber pep.digital vorgegeben wurde. Es verfolgt einen modernen deklarativen Ansatz, der die UI-Entwicklung vereinfacht, da die Benutzeroberfläche direkt in Kotlin beschrieben werden kann.

Glance API

Für die Entwicklung der Widgets kommt die Glance API zum Einsatz. Sie basiert auf Jetpack Compose und verfolgt ebenfalls einen deklarativen Ansatz, was die Entwicklung vereinfacht, da dieselben Konzepte wie im restlichen UI-Code verwendet werden können.

Retrofit 2

Die Netzwerkkommunikation mit dem Backend wird über Retrofit 2 erfolgen. Es ermöglicht, REST-Schnittstellen als Kotlin-Interfaces zu definieren, wodurch Netzwerkaufrufe kürzer und sicherer sind.

Backend

Das Backend wird mit Java und Spring Boot entwickelt und stellt die Schnittstellen für die Android-App bereit.

Spring Boot

Spring Boot wurde von der Hochschule als Backend-Framework vorgegeben und ermöglicht die Anbindung an die Datenbank, die Authentifizierung über Keycloak sowie die Bereitstellung der Schnittstellen für die Android-App.

Datenbanken

Die Datenhaltung erfolgt auf zwei Ebenen, um sowohl Offline-Nutzung als auch geräteübergreifenden Zugriff zu ermöglichen.

Room

Room dient als lokale Datenbank auf Basis von SQLite und ist die Single Source of Truth der Anwendung. Dies ermöglicht eine Offline-Nutzbarkeit. Sofern eine

Internetverbindung vorhanden ist, läuft die Synchronisation mit dem Server nach dem Prinzip Local-First.

PostgreSQL

PostgreSQL wird serverseitig eingesetzt, um Nutzerdaten und Aktivitäten zentral zu speichern. Dadurch können die Daten geräteübergreifend abgerufen und langfristig gesichert werden.

Authentifizierung

Die Authentifizierung erfolgt zentral über Keycloak.

Keycloak

Für die Authentifizierung wird Keycloak eingesetzt. Es wurde von der Hochschule Esslingen vorgegeben und verwaltet die Benutzeridentitäten zentral und unabhängig von der Anwendungslogik.

Externe Schnittstellen

WeatherAPI.com

Über die WeatherAPI.com-Schnittstelle werden per HTTP-Aufruf die Wetterinformationen abgerufen. Die Kommunikation erfolgt über einen API-Key im Backend, welches wetterbasierte Push-Benachrichtigungen ermöglicht.

2.4 Verwendete Schnittstellentechnologie

Die Kommunikation zwischen Frontend, Backend und externen Diensten erfolgt über eine REST-basierte API. Die Daten werden im JSON-Format übertragen und über HTTP-Anfragen zwischen den Systemen ausgetauscht.

Für die Kommunikation zwischen Android-App und Backend wird im Frontend die Bibliothek Retrofit 2 verwendet. Das Spring-Boot-Backend stellt REST-Endpunkte bereit, über die Aktivitäten, Benutzerinformationen und Streak-Daten verarbeitet werden können.

Die Authentifizierung erfolgt über Keycloak mittels OAuth2/OIDC. Nach erfolgreichem Login erhält die mobile App ein Access-Token sowie ein langlebiges Refresh-Token. Während das Access-Token bei allen weiteren API-Anfragen im HTTP-Header zur Autorisierung mitgesendet wird, ermöglicht das Refresh-Token im Hintergrund eine automatische und nahtlose Erneuerung der Nutzersession, ohne dass eine erneute Passworteingabe erforderlich ist.

Zusätzlich kommuniziert das Backend per HTTPS mit WeatherAPI.com, um die Wetterdaten für die wetterbedingten Pushbenachrichtigungen zu erhalten.

Schnittstellenbeschreibung (API)

POST /api/users/login

- Anmeldung eines Nutzers
- Requested Body: { "email": "...", "password": "..." }
- Antwort: { "accessToken": "...", "userId": "UUID", "refreshToken": "..." }

POST /api/users/refresh

- Sitzungs Überprüfung
- Requested Body: { "refreshToken": "..." }
- Antwort: { "accessToken": "...", "userId": "UUID", "refreshToken": "..." }

POST /api/users/register

- Registrierung eines Nutzers
- Requested Body: { "name": "...", "vorname": "...",
"email": "...", "password": "..." }
- Antwort: { "userId": "..." }

PUT /activities/id

- (Upsert) Bestehende Aktivität bearbeiten oder neu hinzufügen
- Request Body: { "activityName": "Spazieren",
"activityDate": "...",
"createdAt": "...",
"userId": "..." }
- Antwort: 201 Created (kein Body)

GET /api/weather

- Wetterdaten abrufen
- Antwort: { "condition": "..." }

GET /api/friends/{userId}

- Freundesliste eines Nutzers abrufen
- Antwort: [{ "userId": "UUID", "username": "...",
"friendCode": "..." }]

POST /api/friends/{userId}/request/{friendCode}

- Freundschaftsanfrage senden
- Request Body: – (nicht erforderlich)
- Antwort: 200 OK (kein Body)

GET /api/friends/{userId}/requests

- Offene Freundschaftsanfragen abrufen
- Antwort: [{ "requestId": "UUID",
"username": "...", "friendCode": "..." }]

POST /api/friends/requests/{requestId}/accept

- Freundschaftsanfrage annehmen
- Request Body: – (nicht erforderlich)
- Antwort: 200 OK (kein Body)

POST /api/friends/requests/{requestId}/decline

- Freundschaftsanfrage ablehnen
- Request Body: – (nicht erforderlich)
- Antwort: 200 OK (kein Body)

DELETE /api/friends/{userId}/remove/{friendId}

- Freund aus der Freundesliste entfernen
- Antwort: 200 OK (kein Body)

2.5 Datenbank

2.5.1 Logisches Datenmodell

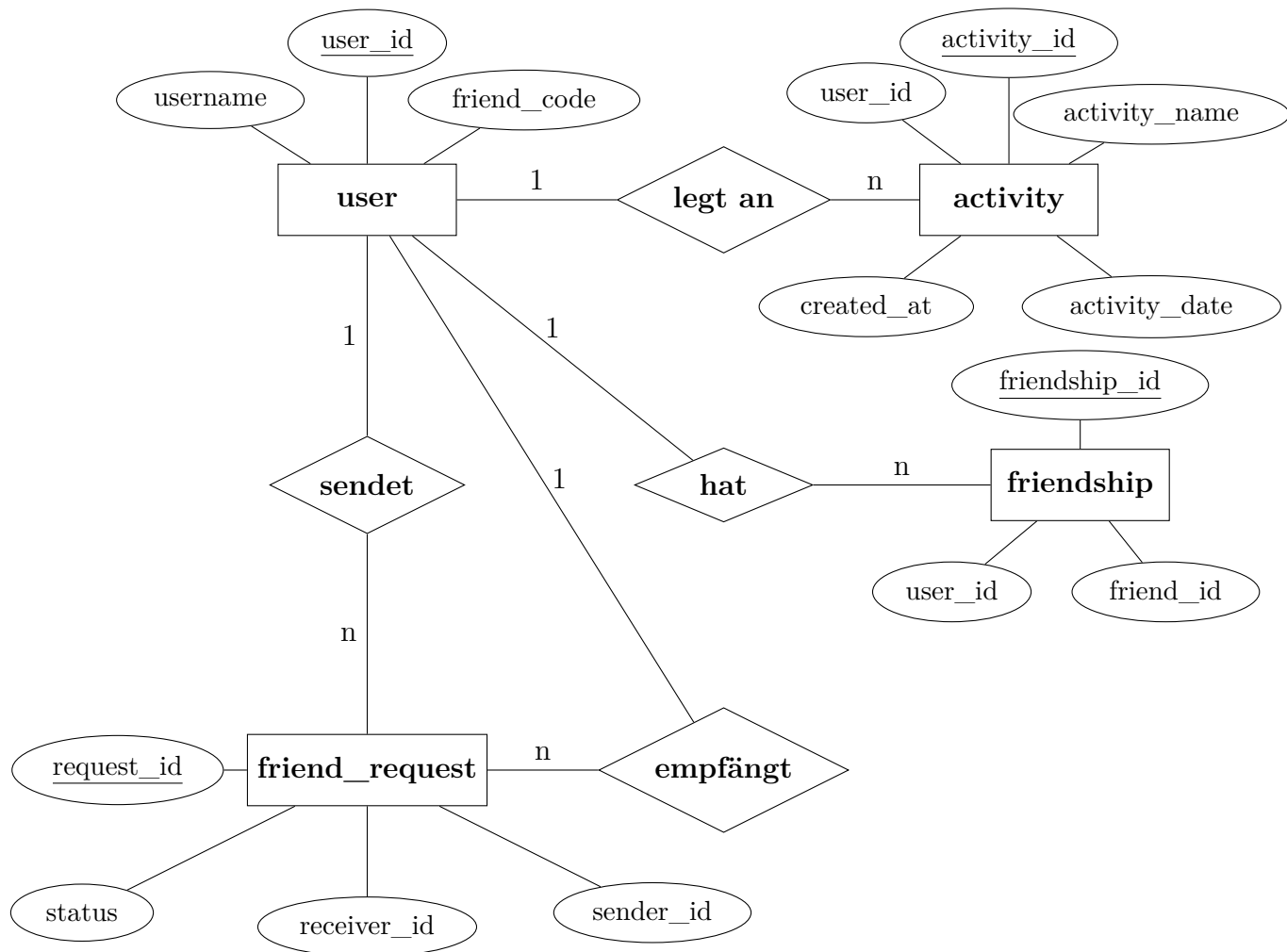


Abbildung 2.5: ER-Modell - Remote-Datenbank

2.5.2 Physisches Datenmodell

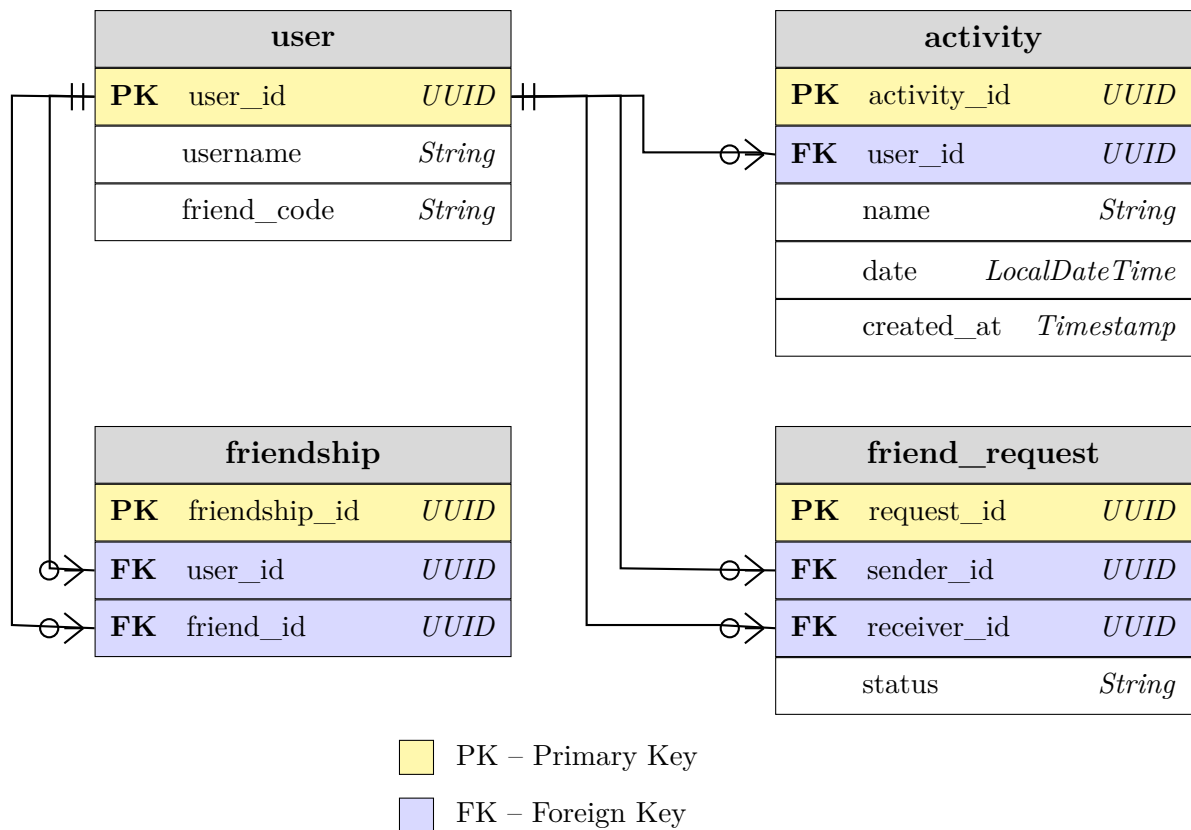


Abbildung 2.6: Physisches Datenmodell – Remote Datenbank

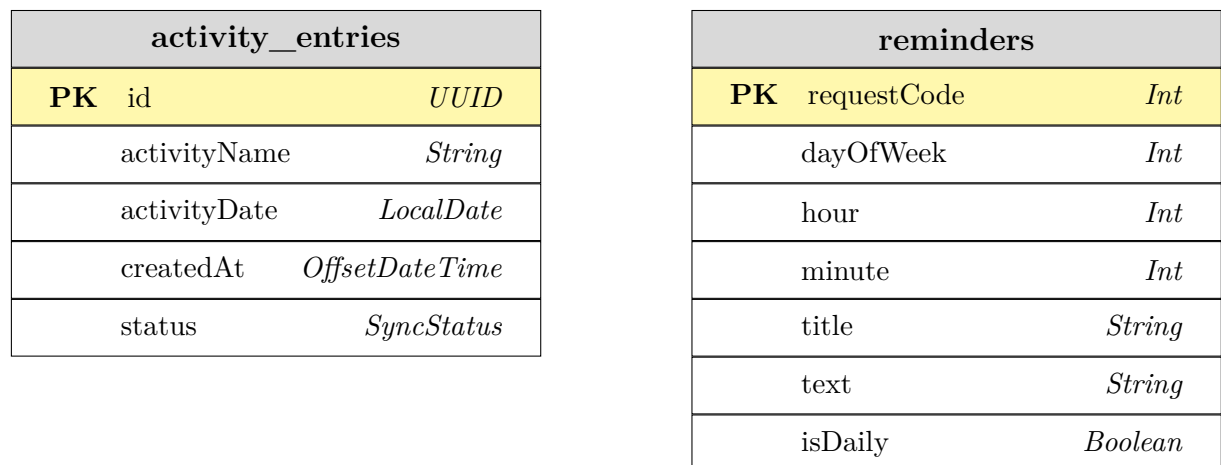


Abbildung 2.7: Physisches Datenmodell – Lokale Datenbank

3 Meilenstein

3.1 Installations- und Administrationshandbuch

3.1.1 Installation für Entwickler

Voraussetzungen

Für die lokale Entwicklung werden folgende Komponenten benötigt:

- Android Studio
- IntelliJ IDEA
- Java Development Kit (JDK 21)
- Maven
- Docker Desktop
- Git
- Android SDK (API Level 36)

Installation

Projekt herunterladen Das Projekt aus dem GitHub-Repository klonen und in Android Studio bzw. IntelliJ IDEA öffnen.

Backend einrichten

1. Docker Desktop installieren und starten.
2. Die Datei `.env.example` in `.env` umbenennen.
3. Die erforderlichen Umgebungsvariablen (z. B. Datenbankname, Benutzername und Passwort) eintragen.
4. Das **EnvFile-Plugin** in IntelliJ IDEA installieren.
5. In den Build-Konfigurationen EnvFile aktivieren und die `.env`-Datei auswählen.
6. Die PostgreSQL-Datenbank starten:

```
docker-compose up -d
```

7. Anschließend `BackendApplication.java` starten.

Datenbank Nach dem Start des Backends kann die Datenbank über das Database-Fenster von IntelliJ IDEA verbunden werden. Nach erfolgreicher Verbindung sollten die vom Projekt verwendeten Tabellen sichtbar sein.

Android-App In Android Studio unter **Build Variants** den gewünschten Build auswählen:

- **localDebug** – Verbindung zum lokal gestarteten Backend
- **serverDebug** – Verbindung zum Backend-Server

Anschließend kann die App auf einem Emulator oder Android-Gerät gestartet werden.

3.1.2 Installation für Endnutzer

Voraussetzungen

Für die Nutzung der Anwendung werden folgende Voraussetzungen benötigt:

- **Betriebssystem:** Android 12 oder höher
- **Freier Speicherplatz:** mindestens 100 MB
- **Internetverbindung** für die Kommunikation mit dem Backend-Server
- **Benachrichtigungsberechtigung** für Erinnerungen
- **Standortberechtigung** für wetterbasierte Funktionen

Die Erfassung lokal gespeicherter Aktivitäten ist auch ohne Internetverbindung möglich.

Installation (laufender Server benötigt)

1. Die APK-Datei aus der Release Sektion aus dem GitHub Repository herunterladen.
2. Die Anwendung starten.
3. Den angeforderten Berechtigungen für Benachrichtigungen und Standort zustimmen.
4. Einen Benutzeraccount erstellen oder anmelden.

Die Anwendung kann anschließend verwendet werden.

3.1.3 Administration

Zur Administration der Anwendung gehören insbesondere:

- Verwaltung der PostgreSQL-Datenbank
- Pflege der Umgebungsvariablen (**.env**)

- Verwaltung der Docker-Container
- Verwaltung der Benutzer über Keycloak
- Wartung des Backend-Server

3.2 Aufteilung des Teams

Die folgende Tabelle zeigt die primäre Zuständigkeit für die einzelnen Komponenten des Aktivitätstrackers.

Daniel Trbara

Authentifizierung: Keycloak-Integration, Login/Register-Screens, Google Login

Aktivitäten: Lokale Aktivitätseinträge in DB abspeichern

DevOps & Orga: CI/CD Pipeline, Bugfixing/Refactoring, Server mieten

Christian Sakru

Benachrichtigungen: Statische/Konfigurierbare Benachrichtigungen, Wetter-API Einbindung, wetterbasierte Benachrichtigungen

Datenbanken: Postgres-Datenbank Setup

DevOps & Orga: Bugfixing/Refactoring

Lionel Toussoumsnaba

Homescreen: Kalenderanzeige, Navigation

Aktivitäten: Aktivitätenliste, Aktivitätsvorschläge

Konzeption: UI/UX Design (Figma)

DevOps & Orga: Bugfixing/Refactoring

Mirac Sancak

Datenbanken: Room-Datenbank Setup, Datenbanken Synchronisation

Aktivitäten: Streak Logik

Statistiken: Statistiken auswerten und anzeigen

DevOps & Orga: Bugfixing/Refactoring, APK erstellen, Server mieten

Abdullah Bilen

Widgets: Aktivitäten hinzufügen- und Streak Widget

Aktivitäten: Streak UI, Aktivitäten bearbeiten

Freunde: Freunde UI, Freunde einladen über ID, Streakanzeige der Freunde

DevOps & Orga: Bugfixing/Refactoring

Alle Teammitglieder

Konzeption: Architektur-Entwurf, Datenmodellierung

3.3 Reflektion Projektmanagement

Im Vergleich zur ursprünglichen Planung hat sich in der finalen Umsetzung nur wenig verändert. Die wesentlichen Abweichungen lassen sich auf folgende Punkte reduzieren:

- **Server-Deployment:** Das Anmieten eines Servers war ursprünglich nicht vorgesehen. Im Projektverlauf haben wir uns jedoch dazu entschieden, das Backend samt Docker-Containern auf einem externen Server zu deployen, da dies die Grundvoraussetzung dafür war, dass der Kunde die App auf seinem eigenen Gerät testen kann. Ohne Server wären lediglich lokale Starts in der Entwicklungsumgebung möglich gewesen.
- **Einstellungsbereich:** Der ursprünglich geplante Einstellungsbereich wurde nicht in vollem Umfang umgesetzt. Letztendlich wurde darin ausschließlich die Konfiguration von Benachrichtigungen realisiert.
- **Aufgabenverteilung:** Zu Beginn war vorgesehen, dass alle Teammitglieder sowohl im Frontend als auch im Backend tätig sind. Im späteren Projektverlauf hat sich eine stärkere Spezialisierung ergeben zwischen den einzelnen Teammitgliedern.

Phasen	geschätzter Aufwand	tatsächlicher Aufwand
Projektmanagement & Vorbereitung		
Sprint-Meetings, Ticket-Refinement & Organisation	210 h	247 h
Anforderungsanalyse, Architektur & UI-Design	160 h	77 h
Implementierung		
Setup (Backend, Keycloak, pipeline, refactoring, Sync, APK, Server)	133 h	177 h
Aktivitäten-Tracking	54 h	50 h
Streak-System	28 h	24 h
Benachrichtigungs-Service & Wetter-API	48 h	56 h
Widget (Glance API)	32 h	28 h
Freunde	25 h	33 h
Statistiken	16 h	20 h
Qualitätssicherung & Abschluss		
Code-Reviews, Bugfixing	Testing	220 h
122		
Dokumentation (MS-Doku, Präsi & Handbuch)	260 h	274
Gesamtaufwand (Team)	geschätzt 1.186 h	1108 h

Tabelle 3.1: Aufwandsschätzung im vergleich mit tatsächlicher Aufwand
aktualisierter geschätzter Aufwand (Ticket schätzung mit dem Kunden)

3.4 Reflektion Lernfortschritt

Im Verlauf des Projekts konnten wir sowohl fachlich als auch organisatorisch wichtige Erfahrungen sammeln. Auf technischer Ebene stand die Entwicklung einer Android-App mit Jetpack Compose sowie die Bereitstellung eines Spring-Boot-Backends mit Remote-Datenbank im Fokus. Ein zentraler Lerneffekt lag in der sauberen Trennung der Projektstrukturen und der Synchronisation zwischen lokaler Room-Datenbank und dem entfernten Server.

Auf organisatorischer Ebene zeigte sich, dass die parallele Arbeit an verschiedenen

Features eine automatisierte Qualitätssicherung erfordert. Durch die Einrichtung einer CI/CD-Pipeline inklusive automatischem APK-Build bei jedem Main-Merge sowie automatisierten Abläufen für Subbranches konnten Integrationstests und Fehlerbehebungen deutlich beschleunigt werden. Die systematische Erfassung von Aufgaben über Tickets (vom ersten Aufsetzen des Grundgerüsts über UI-Anpassungen bis hin zum letzten Refactoring) half dem Team dabei, den Überblick zu behalten, Abhängigkeiten frühzeitig zu erkennen und flexibel auf unerwartete Bugfixes zu reagieren.

3.5 Ausblick

Im Rahmen des Projekts konnten alle definierten MVP-Anforderungen erfolgreich umgesetzt werden, darüber hinaus wurden bereits einige der Extended Goals realisiert. Die folgenden Extended Goals bieten Potenzial für eine zukünftige Weiterentwicklung der Anwendung:

- **Mehrsprachigkeit:** Unterstützung mehrerer Sprachen zur Verbesserung der Zugänglichkeit für internationale Nutzer.
- **Light-/Dark-Theme:** Einführung eines anpassbaren Erscheinungsbilds für eine bessere Nutzererfahrung.
- **Server-zu-Gerät-Synchronisation:** Aktuell werden lokal erstellte Einträge auf dem Server gespeichert. Wird die App jedoch auf einem neuen Gerät installiert oder werden die App-Daten gelöscht, erfolgt keine Wiederherstellung der gespeicherten Daten. Eine bidirektionale Synchronisation würde dieses Problem beheben.
- **Profileinstellungen:** Erweiterung der Nutzereinstellungen, um persönliche Profilinformationen verwalten zu können.
- **Aktivität löschen:** Möglichkeit für Nutzer, bereits erfasste Aktivitäten nachträglich zu entfernen.
- **Account löschen:** Bereitstellung einer Funktion zur vollständigen Löschung des Nutzerkontos inklusive aller zugehörigen Daten.
- **Logout:** Bereitstellung einer Funktion, über die sich Nutzer aktiv aus der Anwendung abmelden können.

Anhang

Das folgende Glossar erläutert die in diesem Dokument verwendeten Fachbegriffe.

Begriff	Erklärung
MVP	Minimum Viable Product – minimaler funktionsfähiger Prototyp
IAM	Identity and Access Management – Verwaltung von Identitäten und Zugriffsrechten
OAuth2	Offenes Autorisierungsprotokoll für sicheren Zugriff auf Ressourcen
OIDC	OpenID Connect – Authentifizierungsschicht auf Basis von OAuth2
REST	Representational State Transfer – Architekturstil für Webservices
API	Application Programming Interface – Schnittstelle zur Kommunikation zwischen Systemen
Room	Googles offizielle Bibliothek für lokale Datenbanken auf Android, basierend auf SQLite
Glance	Jetpack-Bibliothek zur deklarativen Entwicklung von Android Home-Screen-Widgets

A.1 Lizenzen

Dieser Abschnitt dokumentiert die im Projekt eingesetzten Fremdlizenzen sowie die Lizenz, unter der der eigene Quellcode veröffentlicht wird.

A.1.1 Verwendete Frameworks

Frameworks bilden das strukturelle Gerüst der Anwendung und geben den Kontrollfluss vor. Die folgende Tabelle listet die im Projekt eingesetzten Frameworks mit Version und Lizenz auf.

Framework	Version	Zweck	Lizenz
Android SDK	API 36	Android-Plattform-APIs	Android SDK License
Jetpack Compose	1.10.0	Deklaratives UI-Framework	Apache 2.0
Glance AppWidget	1.1.1	Framework für Home-Screen-Widgets	Apache 2.0
Spring Boot	4.0.6	Backend-Framework (REST-API, Konfiguration, Embedded Tomcat)	Apache 2.0
Spring Framework	7.0.7	Core-Container (Beans, Context, AOP)	Apache 2.0
Spring Security	7.0.5	Security-Framework (Filterkette, OAuth2)	Apache 2.0
Hibernate ORM	7.2.12	JPA-Implementierung als ORM-Framework	Apache 2.0
JUnit Jupiter	5.x / 6.0.3	Test-Framework für Android und Backend	EPL 2.0

Tabelle A.2: Verwendete Frameworks

A.1.2 Verwendete Bibliotheken

Bibliotheken werden gezielt aus dem Anwendungscode heraus aufgerufen und ergänzen die Frameworks um spezifische Funktionalitäten. Die folgenden Tabellen listen die direkt eingebundenen Bibliotheken getrennt nach Android-Anwendung und Backend auf.

Android-Anwendung

Bibliothek	Version	Zweck	Lizenz
Kotlin Standard Library	2.1.20	Standardbibliothek der Programmiersprache	Apache 2.0
Kotlin Coroutines	1.7.3	Asynchrone Verarbeitung	Apache 2.0
Compose Material 3	1.4.0	Material-Design-Komponenten	Apache 2.0
Activity Compose	1.13.0	Activity-Integration für Compose	Apache 2.0
Lifecycle / ViewModel	2.10.0	Lifecycle-aware Komponenten, ViewModel	Apache 2.0
Room	2.8.4	Lokale SQLite-Datenbank (Single Source of Truth)	Apache 2.0
WorkManager	2.11.2	Hintergrund-Synchronisation	Apache 2.0
DataStore Preferences	1.1.7	Persistenz von Tokens und User-ID	Apache 2.0
Security Crypto	1.1.0-alpha06	AES-Verschlüsselung (EncryptedSharedPreferences)	Apache 2.0
Retrofit	2.11.0	Typsicherer REST-Client	Apache 2.0
Retrofit Gson Converter	2.11.0	JSON-Serialisierung für Retrofit	Apache 2.0
OkHttp Logging Interceptor	4.12.0	HTTP-Logging während der Entwicklung	Apache 2.0
Gson	2.13.2	JSON-Parser	Apache 2.0
AndroidX Credentials	1.3.0-beta01	Anmeldung mit Credential Manager	Apache 2.0
Google Identity (googleid)	1.1.1	Google-Login	Android SDK License
Play Services Location	21.3.0	Standortdaten für wetterbasierte Erinnerungen	Android SDK License
AndroidSVG	1.4	SVG-Rendering	Apache 2.0
Mockito / Mockito-Kotlin	5.x	Mocking-Bibliothek für Tests	MIT

Tabelle A.3: Direkt eingebundene Bibliotheken der Android-Anwendung

Backend

Bibliothek	Version	Zweck	Lizenz
Java Standard Library	21	Standardbibliothek der Programmiersprache (OpenJDK)	GPL 2.0 + Classpath Exception
Hibernate Validator	9.0.1	Implementierung der Jakarta Bean Validation	Apache 2.0
PostgreSQL JDBC Driver	42.7.10	Datenbanktreiber	BSD-2-Clause
Keycloak Admin Client	24.0.0	Just-in-Time User-Provisionierung in Keycloak	Apache 2.0
Jackson Databind	2.21.2	JSON-Serialisierung	Apache 2.0
Jackson JSR310	2.21.2	JSON-Mapping für <code>java.time</code>	Apache 2.0
Nimbus JOSE + JWT	10.4	JWT-Verarbeitung	Apache 2.0
Lombok	1.18.46	Boilerplate-Reduktion durch Annotationen	MIT
HikariCP	7.0.2	JDBC-Connection-Pool	Apache 2.0
Logback Classic	1.5.32	Logging-Implementierung	EPL 2.0 / LGPL 2.1
SLF4J	2.0.17	Logging-Fassade	MIT
Mockito Core	5.20.0	Mocking-Bibliothek für Tests	MIT
AssertJ Core	3.27.7	Fluent-Assertion-Bibliothek	Apache 2.0

Tabelle A.4: Direkt eingebundene Bibliotheken des Backends

A.1.3 Lizenzkompatibilität

Sämtliche eingesetzten Frameworks und Bibliotheken stehen unter permissiven Open-Source-Lizenzen, überwiegend Apache 2.0, daneben MIT, BSD und EPL. Da diese ausschließlich als Abhängigkeit eingebunden und nicht modifiziert werden, ergeben sich keine Copyleft-Verpflichtungen für den Anwendungscode.

A.1.4 Lizenz des eigenen Codes

Der im Rahmen dieses Projekts entstandene Quellcode wird unter der **MIT-Lizenz** veröffentlicht. Die Lizenz erlaubt die freie Nutzung, Modifikation und Weiterverbreitung des Codes unter der Bedingung, dass der ursprüngliche Copyright-Hinweis erhalten bleibt. Die vollständige Lizenz liegt als **LICENSE**-Datei im Wurzelverzeichnis des Repositories¹.

Die Wahl der MIT-Lizenz wurde getroffen, da sie unter den verbreiteten Open-Source-Lizenzen die geringsten Einschränkungen für eine spätere Weiterverwendung mit sich bringt und mit allen in den Tabellen A.2, A.3 und A.4 Lizenzen kompatibel ist.

¹<https://github.com/hse-classes/ss26-aktivitatstracker/blob/main/LICENSE>

Quellen

1. Figma-Entwürfe, erstellt von Lionel Toussoumsnaba,(05.06.2026).. <https://www.figma.com/design/QJRfYFM1shPvySzpfwocuo/Activity-Tracker?node-id=0-1&t=z3Sr26iHMMWxgfBu-1>
2. WeatherAPI.com (05.06.2026).. <https://www.weatherapi.com>
3. Projekt-Repository auf GitHub (05.06.2026). <https://github.com/hse-classes/ss26-aktivitatstracker>